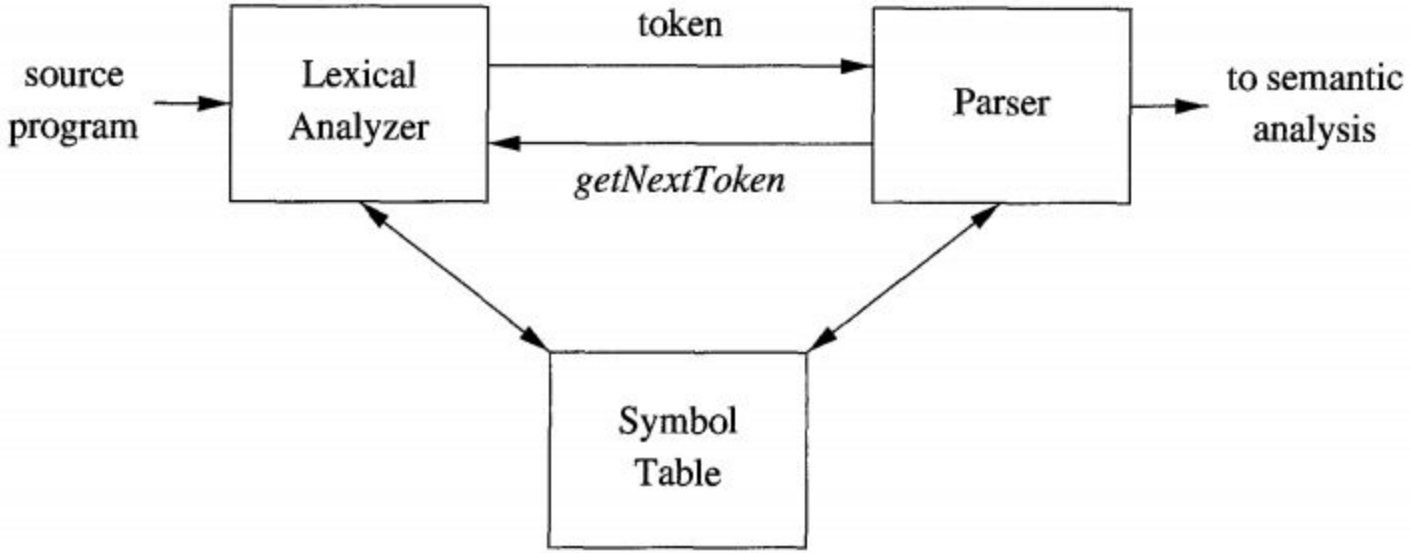
Lexical Analyzer

- **Lexical Analyzer** reads the source program character by character to produce tokens.
- Normally a lexical analyzer doesn't return a list of tokens at one shot, it returns a token when the parser asks a token from it.



Lexical Analyzer

Sometimes, lexical analyzers are divided into a cascade of two processes:

a) *Scanning* consists of the simple processes that do not require tokenization of the input, such as deletion of comments and compaction of consecutive whitespace characters into one.

b) *Lexical analysis* proper is the more complex portion, where the scanner produces the sequence of tokens as output.

LA may also perform certain secondary tasks as the user interface. One such task is striping out from the source program th comments and white spaces in the form of blank, tab and new line characters. Another is correlating error message from the compiler with the source program.

Lexical Analyzer Processes

- **Scanning**
  - to not require input tokenization
  - deletion of comments
  - compaction of consecutive white spaces into one
- **Lexical analysis**
  - to produce sequence of tokens as output

Lexical Analyzer

- The main task of the lexical analyzer is to read the input source program, scanning the characters, and produce a sequence of tokens that the parser can use for syntactic analysis.
- The interface may be to be called by the parser to produce one token at a time
  - Maintain internal state of reading the input program (with lines)
  - Have a function "getNextToken" that will read some characters at the current state of the input and return a token to the parser
- Other tasks of the lexical analyzer include
  - Skipping or hiding whitespace and comments
  - Keeping track of line numbers for error reporting
    - Sometimes it can also produce the annotated lines for error reports
  - Produce the value of the token
  - Optional: Insert identifiers into the symbol table

The Role of the Lexical Analyzer

- to strip out
  - comments
  - whitespaces: blank, newline, tab, …
  - other separators
- to correlate error messages generated by the compiler with the source program
  - to keep track of the number of newlines seen
  - to associate a line number with each error message

Lexical Analysis vs. Parsing

- Simplicity of design
  - Separation of lexical from syntactical analysis -> simplify at least one of the tasks
  - e.g. parser dealing with white spaces -> complex
  - Cleaner overall language design
- Improved compiler efficiency
  - Liberty to apply specialized techniques that serves only lexical tasks, not the whole parsing
  - Speedup reading input characters using specialized buffering techniques
- Enhanced compiler portability
  - Input device peculiarities are restricted to the lexical analyzer

Tokens, Patterns, Lexemes

- Token - pair of:
  - token name – abstract symbol representing a kind of lexical unit
    - keyword, identifier, ...
  - optional attribute value
- Pattern
  - description of the form that the lexeme of a token may take
  - e.g.
    - for a keyword the pattern is the character sequence forming that keyword
    - for identifiers the pattern is a complex structure that is matched by many strings
- Lexeme
  - a sequence of characters in the source program matching a pattern for a token

Example: Tokens, Patterns, and Lexemes

Token	Pattern	Lexeme
if	characters i f	if
else	characters e l s e	else
comparison	< or > or <= or >= or == or !=	<=, !=
id	letter followed by letters and digits	pi, score, D2
number	any numeric constant	3.14, 0, 6.23
literal	anything but “, surrounded by “’s	“core dump”

Attributes for Tokens

- Tokens have at most one associated attribute
- token **id,** e.g., its lexeme, its type, and the location at which it is first found is kept in the symbol table.
- Attribute value for an identifier is a pointer to the symbol-table entry.
- Example

- The token names and associated attribute values for the For tran statement
- `E = M * C ** 2`
- are written below as a sequence of pairs.
- **<id,** pointer to symbol-table entry for **E>**
- **<assign_op>**
- **<id,** pointer to symbol-table entry for M**>**
- **<mult_op>**
- **<id,** pointer to symbol-table entry for **C>**
- **<exp_op>**
- **<number,** integer value 2>

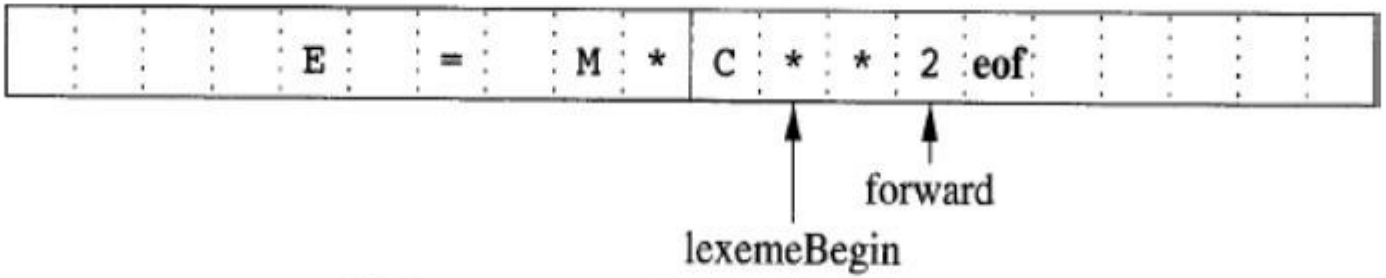
Lexical Errors

- can not be detected by the lexical analyzer alone
  - fi (a == f(x)) …
- lexical analyzer is unable to proceed
  - none of the patterns matches any prefix of the remaining input
  - "panic mode" recovery strategy
    - delete one/successive characters from the remaining input
    - insert a missing character into the remaining input
    - replace a character
    - transpose two adjacent characters

Input Buffering

- How to speed the reading of source program ?
- to look one additional character ahead
- e.g.
  - to see the end of an **identifier** you must see a character
    - which is not a letter or a digit
    - not a part of the lexeme for **id**
  - in C
    - - ,= , <
    - ->, ==, <=
- two buffer scheme that handles large lookaheads safely
- sentinels – improvement which saves time checking buffer ends

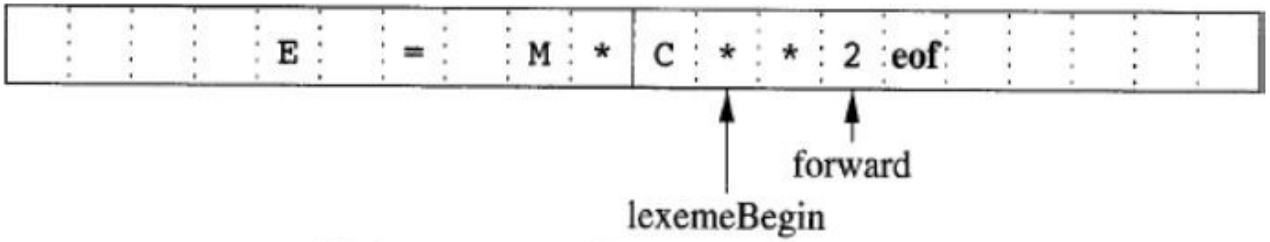
Buffer Pairs



Using a pair of input buffers

- Advancing *forward* requires that we first test whether we have reached the end of one of the buffers.
- If so, we must reload the other buffer from the input. And move *forward* to the beginning of the newly loaded buffer.

Sentinels



Using a pair of input buffers

- If we use the previous scheme as described, we must check, each time we advance forward, that we have not moved off one of the buffers.
- If we do, then we must also reload the other buffer.
- Thus, for each character read, we make two tests:
  - one for the end of the buffer.
  - And one to determine what character is read (the latter may be a multiway branch).

Sentinels

- *forward* pointer
  - to test if it is at the end of the buffer
  - to determine what character is read (multiway branch)
- sentinel
  - added at each buffer end
  - can not be part of the source program
  - character **eof** is a natural choice
    - retains the role of entire input end
    - when appears other than at the end of a buffer it means that the input is at an end

Specification of Tokens

- Regular expressions are an important notation for specifying lexeme patterns.
- Some Terminology:

- **Alphabet**: a finite set of symbols. **Ex:** letters, digits, punctuation, set {0, 1} is the binary alphabet, ASCII characters is an alphabet.
- **String** :
  - Finite sequence of symbols on an alphabet
  - Sentence and word are also used in terms of string
  - ε is the empty string
  - |s| is the length of string s.
- **Language**: sets of strings over some fixed alphabet
  - ∅ the empty set is a language.
  - {ε} the set containing empty string is a language
  - The set of well-formed C programs is a language
  - The set of all possible identifiers is a language.

Terms for Parts of Strings

The following string-related terms are commonly used:

1. A *prefix* of string s is any string obtained by removing zero or more symbols from the end of s. For example, **ban**, **banana**, and ϵ are prefixes of **banana**.

2. A *suffix* of string s is any string obtained by removing zero or more symbols from the beginning of s. For example, **nana**, **banana**, and ϵ are suffixes of **banana**.

3. A *substring* of s is obtained by deleting any prefix and any suffix from s. For instance, **banana**, **nan**, and ϵ are substrings of **banana**.

4. The *proper* prefixes, suffixes, and substrings of a string s are those, prefixes, suffixes, and substrings, respectively, of s that are not ϵ or not equal to s itself.

5. A *subsequence* of s is any string formed by deleting zero or more not necessarily consecutive positions of s. For example, **baan** is a subsequence of **banana**.

Terms for Parts of Strings

The following string-related terms are commonly used:

- A *prefix* of string *s* is any string obtained by removing zero or more symbols from the end of *s*. For example, **ban, banana,** and **ε** are prefixes of **banana.**

- A *suffix* of string *s* is any string obtained by removing zero or more symbols from the beginning of *s*. For example, **nana, banana,** and **ε** are suffixes of **banana.**

- A *substring* of *s* is obtained by deleting any prefix and any suffix from *s*. For instance, **banana, nan,** and **ε** are substrings of **banana.**

- The *proper* prefixes, suffixes, and substrings of a string *s* are those, prefixes, suffixes, and substrings, respectively, of *s* that are not ε or not equal to *s* itself.

- A *subsequence* of *s* is any string formed by deleting zero or more not necessarily consecutive positions of *s*. For example, **baan** is a subsequence of **banana.**

Operators on Strings

- Concatenation
  - If x and y are strings, then the *concatenation* of x and y, denoted xy, is the string formed by appending y to x.
  - Example, if x = dog and y = house, then xy = doghouse.
  - The empty string is the identity under concatenation; that is, for any strings, $\varepsilon s = s\varepsilon = s$.
- Exponentiation
  - Define s^0 to be ε, and for all $i > 0$, define s^i to be $s^{i-1}s$. Since $\varepsilon s = s$, it follows that $s^1 = s$. Then $s^2 = ss$, $s^3 = sss$, and so on.

Operations on Languages

OPERATION	DEFINITION AND NOTATION
Union of L and *M*	$L \cup M = \{s \mid s \text{ is in } L \text{ or } s \text{ is in } M\}$
Concatenation of *L* and *M*	$LM = \{st \mid s \text{ is in } L \text{ and } t \text{ is in } M\}$
Kleene closure of *L*	$L^* = \bigcup_{i=0}^{\infty} L^i$
Positive closure of *L*	$L^+ = \bigcup_{i=1}^{\infty} L^i$

Operations on Languages

- Concatenation:
  - $L_1L_2 = \{ s_1s_2 \mid s_1 \in L_1 \text{ and } s_2 \in L_2 \}$
- Union
  - $L_1 \cup L_2 = \{ s \mid s \in L_1 \text{ or } s \in L_2 \}$
- Exponentiation:
  - $L^0 = \{\varepsilon\}$ $L^1 = L$ $L^2 = LL$
- Kleene Closure
  - $L^* = \bigcup_{i=0}^{\infty} L^i$
- Positive Closure
  - $L^+ = \bigcup_{i=1}^{\infty} L^i$

Example

- $L_1 = \{a,b,c,d\}$ $L_2 = \{1,2\}$
- $L_1L_2 = \{a1,a2,b1,b2,c1,c2,d1,d2\}$
- $L_1 \cup L_2 = \{a,b,c,d,1,2\}$
- L_1^3 = all strings with length three (using a,b,c,d}
- L_1^* = all strings using letters a,b,c,d and empty string
- L_1^+ = doesn't include the empty string

Example

L={A, B, ... , Z, a, b, ... , z} and D={0, 1, ... 9}.

- L U D is the set of letters and digits - strictly speaking the language with 62 strings of length one, each of which strings is either one letter or one digit.
- LD is the set of 520 strings of length two, each consisting of one letter followed by one digit.
- L^4 is the set of all 4-letter strings.
- L^* is the set of all strings of letters, including λ, the empty string.
- $L(L \cup D)^*$ is the set of all strings of letters and digits beginning with a letter.
- D^+ is the set of all strings of one or more digits.

Regular Expressions

- We use regular expressions to describe tokens of a programming language.
- A regular expression is built up of simpler regular expressions (using defining rules)
- Each regular expression denotes a language.
- Regular expression ***r*** denotes a language *L(r),* which also recursively defined by languages denoted by ***r'***s subexpressions.
- A language denoted by a regular expression is called as a **regular set**.

Ex: Regular expression of C identifiers is

*letter_* (*letter_* | *digit*)*

Induction: Larger regular expressions from smaller ones.

- There are two rules that form the basis:

  1. ε is a regular expression, and L(ε) is {ε}, that is, the language whose sole member is the empty string.

  2. If *a* is a symbol in ∑ then *a* is a regular expression, and L(***a***) = {*a*}, that is, the language with one string, of length one, with a in its one position.

There are four parts to the induction

Suppose *r* and *s* are regular expressions denoting languages *L(r)* and *L(s)*. Then

1. *(r)*| (*s)* is a regular expression denoting the language *L* (*r)* U *L* (*s)*.
2. *(r)(s)* is a regular expression denoting the language *L(r)L(s)*.
3. *(r)** is a regular expression denoting (*L(r)*)*.
4. *(r)* is a regular expression denoting *L(r)*.

This last rule says that we can add additional pairs of parentheses around expressions without changing the language they denote.

Regular Expressions (Rules)

Regular expressions over alphabet Σ

Reg. Expr	Language it denotes
ε	$\{\varepsilon\}$
$a \in \Sigma$	$\{a\}$
$(r_1) \mid (r_2)$	$L(r_1) \cup L(r_2)$
$(r_1)\,(r_2)$	$L(r_1)\,L(r_2)$
$(r)^*$	$(L(r))^*$
(r)	$L(r)$

- $(r)^+ = (r)(r)^*$
- $(r)? = (r) \mid \varepsilon$

Regular Expressions (cont.)

- We may remove parentheses by using precedence rules.
  - * highest
  - concatenation next
  - | lowest
- $ab^*|c$ means $(a(b)^*)|(c)$

- Ex:
  - $\Sigma = \{0,1\}$
  - $0|1 \Rightarrow \{0,1\}$
  - $(0|1)(0|1) \Rightarrow \{00,01,10,11\}$
  - $0^* \Rightarrow \{\varepsilon,0,00,000,0000,....\}$
  - $(0|1)^*$ => all strings with 0 and 1, including the empty string

Example

Let = *{a,b}*.

1. The regular expression **a|b** denotes the language {*a, b*}.

2. **(a|b)(a|b)** denotes *{aa,ab, ba, bb},* the language of all strings oflength two over the alphabet - Another regular expression for the same language is **aa|ab|ba|bb.**

3. **a*** denotes the language consisting of all strings of zero or more a's, that is, {ε, *a, aa, aaa,...* }.

4. **(a|b)*** denotes the set of all strings consisting of zero or more instances of *a or b,* that is, all strings of *a*'s and *b*'s: {ε, *a, b, aa, ab, ba, bb, aaa, ...* }. Another regular expression for the same language is **(a*b***)*.

5. **a|a*b** denotes the language {*a, b, ab, aab, aaab, ...* }, that is, the string *a*
   and all strings consisting of zero or more a's and ending in *b.*

Algebraic laws for regular expressions

LAW	DESCRIPTION
$r\|s = s\|r$	$\|$ is commutative
$r\|(s\|t) = (r\|s)\|t$	$\|$ is associative
$r(st) = (rs)t$	Concatenation is associative
$r(s\|t) = rs\|rt;\ (s\|t)r = sr\|tr$	Concatenation distributes over $\|$
$\epsilon r = r\epsilon = r$	ϵ is the identity for concatenation
$r^* = (r\|\epsilon)^*$	ϵ is guaranteed in a closure
$r^{**} = r^*$	$*$ is idempotent

Regular Definitions

- To write regular expression for some languages can be difficult, because their regular expressions can be quite complex. In those cases, we may use *regular definitions*.
- We can give names to regular expressions, and we can use these names as symbols to define other regular expressions.

- A ***regular definition*** is a sequence of the definitions of the form:

$d_1 \rightarrow r_1$ where d_i is a distinct name and

$d_2 \rightarrow r_2$ r_i is a regular expression over symbols in

. $\Sigma \cup \{d_1, d_2, ..., d_{i-1}\}$

$d_n \rightarrow r_n$

basic symbols previously defined names

Regular Definitions (cont.)

- Ex: Identifiers in C

  letter → A | B | ... | Z | a | b | ... | z

  digit → 0 | 1 | ... | 9

  id → letter (letter | digit) *

  – If we try to write the regular expression representing identifiers without using regular definitions, that regular expression will be complex.

    (A|...|Z|a|...|z) ((A|...|Z|a|...|z) | (0|...|9)) *

- Ex: Unsigned numbers in C

  digit → 0 | 1 | ... | 9

  digits → digit $^{+}$

  opt-fraction → (. digits) ?

  opt-exponent → (E (+|-)? digits) ?

  unsigned-num → digits opt-fraction opt-exponent

Extensions of Regular Expressions

- *One or more instances*. For unary, postfix operator

i. $r^* = r^+ \mid \varepsilon$

ii. $r^+ = rr^* = r^*r$

- *Zero or one instance.* The unary postfix operator ? "zero or one occurrence."

i. *r? => r|ε, or*

ii. *L(r?) => L(r) ∪ {ε}.*

- *Character classes.*

i. $a_1|a_2|...|a_n => [a_1a_2...a_n]$

ii. a|b|c=>[abc]

iii. a|b|c|...|z=>[a-z]

Example (Shorthands)

- Ex: Identifiers in C

  *letter_* → [A-Za-z_]

  digit → [0-9]

  id → *letter_(letter* | *digit)* *

- Ex: Unsigned numbers in C

  digit → [0-9]

  digits → *digit* $^{+}$

  number → *digits (. digits)* ? (E (+|-)? *digits)* ?

Recognition of Tokens

- There are two rules that form the basis:
  1. ε is a regular expression, and L(ε) is {ε}, that is, the language whose sole member is the empty string.
  2. If *a* is a symbol in ∑ then *a* is a regular expression, and L(***a***) = {*a*}, that is, the language with one string, of length one, with a in its one position.

There are four parts to the induction

Suppose *r* and *s* are regular expressions denoting languages *L(r)* and *L(s)*. Then

1. *(r)*|*(s)* is a regular expression denoting the language *L (r)* U *L (s)*.
2. *(r)(s)* is a regular expression denoting the language *L(r)L(s)*.
3. *(r)** is a regular expression denoting *(L(r))**.
4. *(r)* is a regular expression denoting *L(r)*.

This last rule says that we can add additional pairs of parentheses around expressions without changing the language they denote.

Patterns for tokens

digit **[0-9]**

digits → *digit*+

number → *digits* (.*digits*)? (E[+-]? *digits*)?

letter → **[A-Za-z]**

id → *letter* (*letter* | *digit*)*

if → **if**

then → **then**

else → **else**

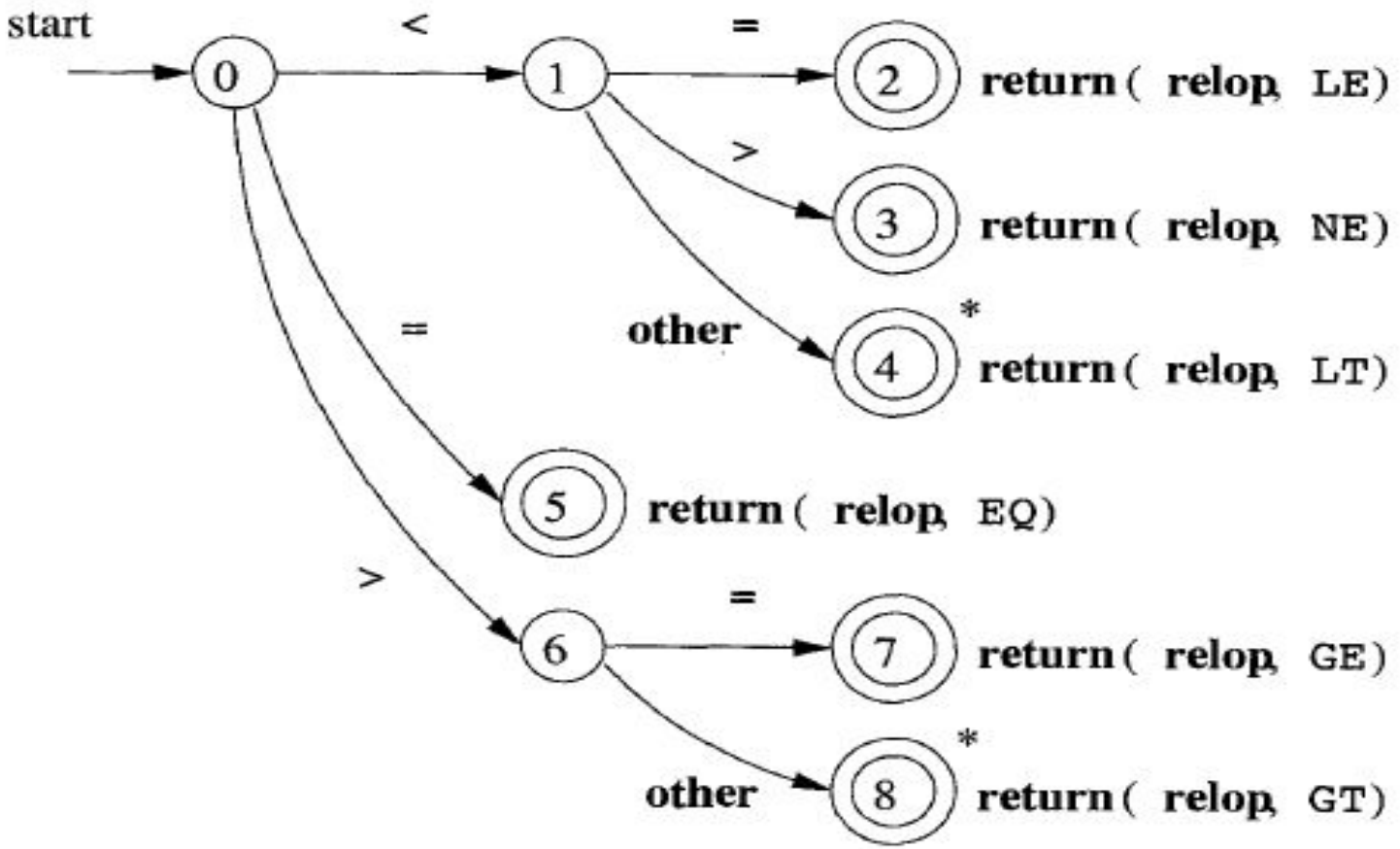
relop → < | > | <= | >= | = | <>

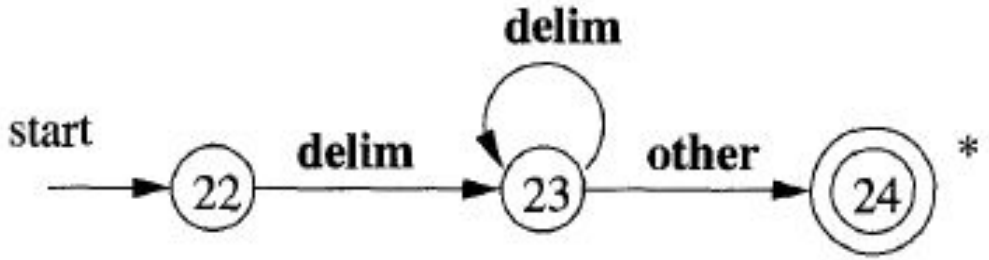
• *ws* → (**blank | tab | newline**)+

Tokens, their patterns, and attribute values

LEXEMES	TOKEN NAME	ATTRIBUTE VALUE
—	—	—
Any *ws*	—	—
`if`	**if**	—
`then`	**then**	—
`else`	**else**	—
Any *id*	**id**	Pointer to table entry
Any *number*	**number**	Pointer to table entry
<	**relop**	LT
<=	**relop**	LE
=	**relop**	EQ
<>	**relop**	NE
>	**relop**	GT
>=	**relop**	GE

Transition diagram for relop





Transition diagram.....

